

Jest do zero

+ 3 desafios simples de teste

Material prático para começar com um exemplo completo e, depois, escrever testes sem copiar o código.

Objetivo: entender o ciclo básico do Jest e depois criar seus próprios testes para três funções simples.

1. Veja	2. Entenda	3. Crie	4. Teste
um exemplo completo	PASS / FAIL	seus testes	e explique o resultado

Regra deste material: só o primeiro teste vem pronto. Nos três desafios seguintes, você recebe apenas a função. O arquivo de teste será criado por você.

1. Preparando o ambiente

Use o Codespace ou um computador com Node.js e npm instalados.

Confira o ambiente

```
node -v  
npm -v
```

Crie o projeto

```
mkdir aula-jest  
cd aula-jest  
npm init -y  
npm install --save-dev jest
```

- | | |
|---|---|
| 1 | npm init -y cria o arquivo package.json . |
| 2 | package.json guarda informações, dependências e comandos do projeto. |
| 3 | --save-dev registra o Jest como dependência de desenvolvimento. |
| 4 | npx jest executa o Jest instalado no projeto. |

Antes de continuar: rode `npx jest --version`. Se aparecer uma versão, o Jest está pronto.

2. Primeiro teste - exemplo completo

Aqui você verá a função e o arquivo de teste. Depois disso, você fará sozinho.

Arquivo: soma.js

```
function somar(a, b) {  
  return a + b;  
}  
  
module.exports = somar;
```

Arquivo: soma.test.js

```
const somar = require('./soma');  
  
test('2 + 3 deve ser 5', () => {  
  expect(somar(2, 3)).toBe(5);  
});
```

test(...)	cria e descreve um teste
expect(...)	executa o valor que será verificado
toBe(5)	define o resultado exato esperado

```
npx jest  
# ou somente este arquivo:  
npx jest soma.test.js
```

3. PASS e FAIL

Antes de executar, tente prever o resultado.

PASS	O resultado obtido é igual ao resultado esperado.
FAIL	O resultado obtido é diferente do esperado.

Faça o teste falhar de propósito

```
expect(somar(2, 3)).toBe(10);
```

Leia a mensagem: Expected é o valor esperado. Received é o valor realmente recebido.

Pequena verificação

1	Volte para toBe(5) e confirme o PASS.
2	Troque para toBe(10) e confirme o FAIL.
3	Explique em uma frase por que o teste falhou.

4. Desafio 1 - dobro de um número

Agora não há código de teste pronto. Crie o arquivo **dobro.test.js**.

```
function dobro(numero) {  
  return numero * 2;  
}  
  
module.exports = dobro;
```

Sua tarefa

1	Importe a função dobro .
2	Crie um teste para dobro(4) .
3	Crie outro teste com um valor diferente.
4	Faça um dos testes falhar de propósito. Depois corrija.

Checkpoint: mostre o arquivo de teste e explique o que significa o valor dentro de **toBe(...)**.

Pense: além de 4, qual valor simples seria interessante testar? E o número 0?

5. Desafio 2 - maior de idade

Crie o arquivo `maiorDeIdade.test.js`. A função agora retorna **true** ou **false**.

```
function maiorDeIdade(idade) {  
  return idade >= 18;  
}  
  
module.exports = maiorDeIdade;
```

Sua tarefa

- | | |
|---|---|
| 1 | Crie um teste para uma idade claramente menor que 18. |
| 2 | Crie um teste para uma idade claramente maior que 18. |
| 3 | Teste exatamente 18 . |
| 4 | Compare 17, 18 e 19. Qual deles é o caso mais importante? |

Checkpoint: explique por que testar exatamente 18 é mais importante que testar apenas 25.

Ideia importante: 18 é a fronteira da regra. Bons testes não usam apenas valores aleatórios; eles procuram pontos onde o comportamento muda.

6. Desafio 3 - situação do aluno

Crie o arquivo `situacaoAluno.test.js`. Agora a função possui três resultados possíveis.

```
function situacaoAluno(nota) {  
  if (nota >= 7) {  
    return 'Aprovado';  
  }  
  
  if (nota >= 5) {  
    return 'Recuperação';  
  }  
  
  return 'Reprovado';  
}  
  
module.exports = situacaoAluno;
```

Sua tarefa

- | | |
|---|--|
| 1 | Crie um teste que resulte em Aprovado . |
| 2 | Crie um teste que resulte em Recuperação . |
| 3 | Crie um teste que resulte em Reprovado . |
| 4 | Depois teste valores próximos das mudanças: 4.9, 5, 6.9 e 7. |

Checkpoint: mostre pelo menos três testes passando e diga quantos comportamentos diferentes da função você testou.

Desafio extra: sem alterar a função, escreva um teste que você acredita que deveria passar, mas que revele uma regra não definida. Ex.: nota negativa ou maior que 10. Discuta: a função deveria aceitar isso?

7. Fechamento - o que você precisa saber

1	Sei dizer o que é o Jest : um framework de testes para JavaScript.
2	Sei criar um arquivo com final .test.js .
3	Sei usar test() , expect() e toBe() .
4	Sei executar todos os testes com npx jest .
5	Sei interpretar PASS , FAIL , Expected e Received .
6	Consigo criar casos de teste sem copiar um exemplo pronto.
7	Consigo pensar em casos de fronteira, como 17/18/19 ou 4.9/5/6.9/7.

Mini desafio final

Escolha uma das três funções. Apague seu arquivo de teste e tente recriá-lo do zero, sem olhar os testes anteriores. Se conseguir, você já dominou a estrutura básica.

Estrutura mental

